

Wrapper Classes in Java



- ✓ Convert primitives to objects
- ✓ Use collections (ArrayList, etc.)
- ✓ Support Java features (annotations, serialization, etc.)

PAGE INDEX (Wrapper Classes in Java PDF)

★ SECTION 1 — Introduction (Pages 1–4)

Page 1 — Cover Page

Page 2 — Index Page

Page 3 — What is a Wrapper Class?

Page 4 — Why Do We Need Wrapper Classes?

★ SECTION 2 — Autoboxing / Unboxing (Pages 5–8)

Page 5 — Autoboxing (Basic + Daily Life Example)

Page 6 — Unboxing (Basic + Daily Life Example)

Page 7 — Autoboxing vs Unboxing (Comparison)

Page 8 — Autoboxing/Unboxing Common Problems

★ SECTION 3 — All 8 Wrapper Classes (Pages 9–15)

Page 9 — Integer Wrapper Class

Page 10 — Short Wrapper Class

Page 11 — Byte Wrapper Class

Page 12 — Long Wrapper Class

Page 13 — Float Wrapper Class

Page 14 — Double Wrapper Class

Page 15 — Boolean Wrapper Class

★ SECTION 4 — Character Wrapper Class (Pages 16–23)

Page 16 — Character Class (Basics)

Page 17 — ASCII & Unicode Basics

Page 18 — Character Case Methods (Upper/Lower/Title)

Page 19 — Character Digit Methods (isDigit, getNumericValue...)

Page 20 — Character Alphabet Methods (isLetter, isAlphabetic...)

Page 21 — Whitespace Methods (isWhitespace, isSpaceChar...)

Page 22 — Identifier Rules (isJavaIdentifierStart/Part)

Page 23 — Unicode Category + Title Case + Type()

★ SECTION 5 — Important Wrapper Methods (Pages 24–30)

Page 24 — `parseXXX()` Methods

Page 25 — `valueOf()` Method (Basic Concepts)

Page 26 — `toString()` Method

Page 27 — `equals()` Method

Page 28 — `compareTo()` Method

Page 29 — `MAX_VALUE` & `MIN_VALUE`

Page 30 — Wrapper Class Immutability

★ SECTION 6 — Advanced Topics (Pages 31–40)

Page 31 — Autoboxing Performance

Page 32 — Memory Behavior of Wrapper Classes

Page 33 — Integer Cache (`valueOf()` Caching Logic)

Page 34 — Wrapper Errors (`NullPointerException` + Conversion issues)

Page 35 — Unicode, Case & Identifier Rules (Advanced)

Page 36 — Unboxing Methods (in-depth)

Page 37 — Advanced Autoboxing Rules

Page 38 — `instanceof` with Wrapper Classes

Page 39 — `equals` vs `==` (Deep Explanation)

Page 40 — `compareTo()` Method (Full Understanding)

Page 41 — Cheatsheet

What is a Wrapper Class?

A wrapper class is a special class in Java that wraps a primitive data type (int, long, etc.) in an object.

This “wraps” the value in a protective box but still allows you to access it when needed.

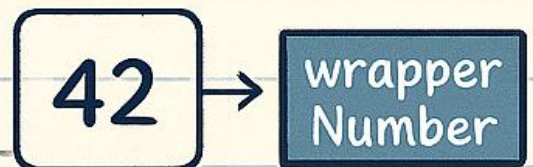


Real-Life Example

Imagine a gift wrapped in a box with a ribbon. The box protects the gift inside, but you can open it when you want to use it.

Code Example

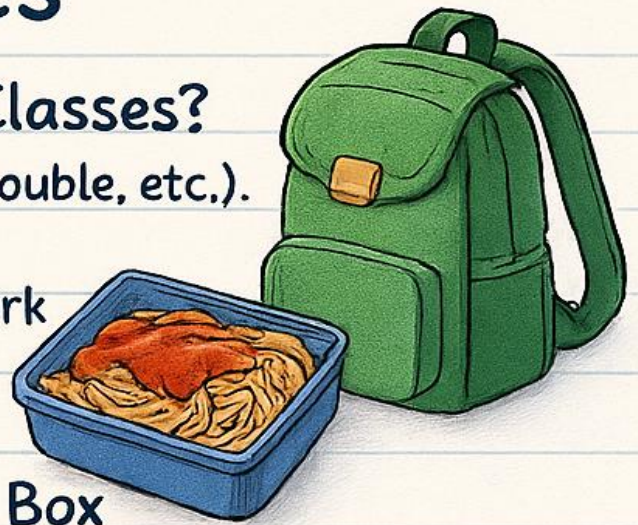
```
int number = 42;  
Integer wrapperNumber = new Integer(number)
```



Why Java Needs Wrapper Classes

Why do we need Wrapper Classes?

- Java has 8 primitive types (int, double, etc.). They are fast but not objects.
- Many powerful Java features work only with objects, so Java created Wrapper Classes.



Daily-Life Example – Tiffin Box

- Think of food as a primitive value (simple, raw, basic).
- But you cannot carry loose food in your hand.
 - Food = primitive
 - Tiffin box = wrapper class
- Just like a tiffin lets you 'carry' food easily...

➤ Tiffin box = wrapper class



🐛 Wrapper Classes let Java 'carry' primitive values into places where only objects are allowed.

🔗 Code Example

- Primitives can't be **null**.
- Wrappers can be null
Useful for optional values (like "marks not entered yet")

+ Utility methods



- Wrapper classes provide useful helper methods.
- Primitive = food
- Collection = school bag
- Java features = places that require tiffin, not loose food)



```
Integer n = null;  
int x = n? NullPointerException  
// Fix  
int x = (n == null) ? 0 : n.  
Integer auto = price;  
    autoboxing (automatic)  
int unboxed = auto
```

Common Pitfall

Primitive = food
Wrapper = tiffin box
Collection = school bag (bag
Java features = places that
require tiffin, not loose food'

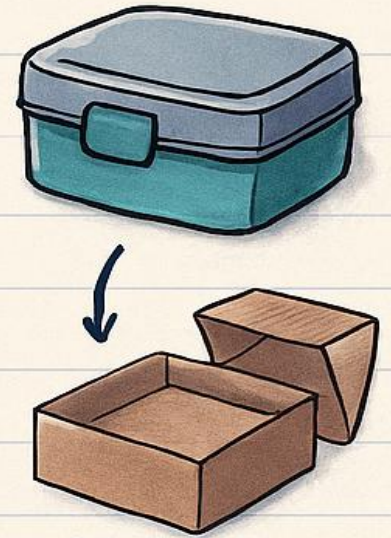
Wrappers exist so Java can 'carry' primitive values into object-only areas.

Autoboxing and Unboxing

What is Autoboxing?

- Autoboxing means Java automatically converts a primitive value into its corresponding wrapper class object.

You don't need to write `Integer.valueOf()` manually – the compiler does it for you.



✓ Example:

```
int num = 10;  
Integer boxed = num;  
    // autoboxing (automatic)
```

```
Integer age = 20;  
int unboxedAge = age;  
    // unboxing
```

Daily-Life Example – Tiffin Box (Automatic packing)

This helps you remember instantly.

- Tiffin packs your food → Autoboxing
- You take food out → Unboxing

This helps you remember instantly.

⚠ Important warning:

Unboxing a null wrapper causes a runtime error:

```
Integer n = null;  
    // ❌ NullPointerException:
```

Automontan 'Example (with comments)

```
Integer n = null;  
// int x = n ❌ NullPointerException
```

✓ Safe version:

```
int x = (n == null) ? 0 : n.
```

Why Autoboxing & Unboxing Matter

- Makes code easier to write
- Lots you use primitives inside collections



Small memory shortcut:

- 'Auto' = Java will automatically wrap
- 'Un' = Java will undo the wrapping

Primitive Types vs Wrapper Classes

- Why compare them?

- Java gives us two ways to store values:
(int, double, char, boolean..)
- Wrapper classes
- Objects that wrap primitive inside

Understanding the difference helps you choose the right one at the right time.

✓ Daily-Life Example - Pencil vs Pencil Box

- Primitive
- Stored in
• Stack
- Very fast
- Does only one job
- No extra features



✓ Null Support

- Cannot store null
 - Useful for 'value not available yet'
- Pencil = primitive (simple value)
→ Pencil box = wrapper (object)



Pencil vs. Pencil Box

- | | |
|-------------------------------------|--|
| • Can hold pencil | • Support labels stickers, storage |
| • Extra space for other items | → Pencil box = wrapper object with extra features) |
| → Pencil = primitive (simple value) | |

✓ Use in Collections

Collections require objects, not primitives:

✗ `ArrayList<int>`

✓ `ArrayList<Integer>`

→ `Integer qty = null`
`int q = qty // auto`

⚠ Important:

Unboxing null causes `NullPointerException`

When to Prefer Primitives vs Wrappers

Choosing between a primitive and a wrapper class is important for writing efficient, clean, and bug-free Java code.

We understand both, now let's see which one is better and when.

Daily-Life Example – Fruits vs Fruit Basket



• Primitive

• Fruit basket

• Can hold fruit

• Can store **null**

• Can be balabeld (empty basket = no fruit)

→ Carrying laose fruits prrimitive

➤ Carrying u loose' (primitive) is fast 

➤ But a basket (wrapper) is needed when you want more features

🔍 Prefer Primitives When...

⚡ Use primitive types (int, 'double, boolean, etc.) when you want:

⚡ High performance
Primitives are VERY fast.

🌀 Lower memory usage
Primitive values take less space

🚧 Value wull never' null
= use when storing, labeling

② Prefer Wrappers When...

Use wrapper classes (Integer, Double, Boolean, etc.) when you need:

```
List<Integer> maks =  
    new hrrayList>(),  
marks:edd(25)  
{ // OK cuserefast
```

⚠ Always check for null:
`int x = (n == null) ? 0 : n.`

🛒 Quick Memory Trick

Autoboxing & Unboxing in Java



What is Autoboxing?

Autoboxing means Java → automatically converts a primitive → wrapper object.

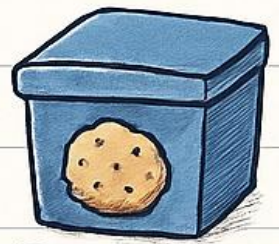
Example `int n = 10;`

`Integer obj = n; // autoboxing`

You don't have to write:

`Integer obj = Integer.valueOf(n)`

Java does it for you!



Biscuit + Box
inside a box



Plain
biscuit

What is Unboxing?

Unboxing means Java → automatically converts a wrapper → primitive,

Example `Integer obj = 20;`

`int n = obj; // unboxing`

Java internally does: `int n = obj.intValue();`



Why Does Java Do This Automatically?

- Easy use primitives $\equiv$ null, unboxing will crash:

`Integer a = null;`
`list.add(5);`

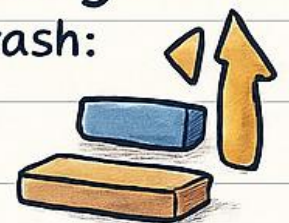


autoboxing

- Cleaner code

No need to manually call methods like `intValue()`.

- Works smoothly with Generics, Streams, Frameworks



Autoboxing



Unboxing Warning

If your wrapper is null, unboxing will crash:
Easy to remember for null values! Unboxing.

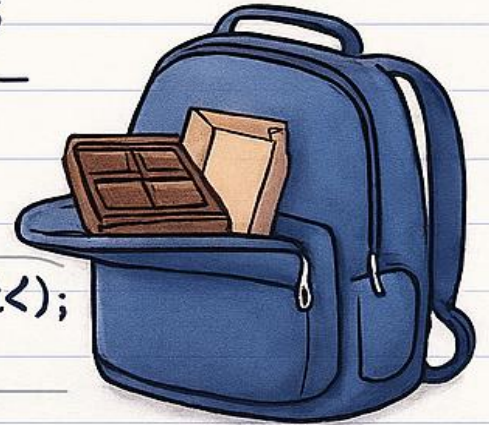
Where Do We Use Wrapper Classes in Java?

1. Wrapper Classes in Collections

- Collections cannot store primitives – they only store objects.

So wrapper classes are required.

```
List<Integer> numbers = new ArrayList<>;  
numbers.add(10);
```



2. Parsing Strings into Numbers

- Wrapper classes provide methods like: `Integer.parseInt()`, `Double.parseDouble()`

- Example:

```
String price = "250";
```

```
int amount = Integer.parseInt(price);
```



3. Using With Characters

- Character wrapper provide helpful methods:

- `Integer.parseInt()`
- `Double.parseDouble()`

6. Using wrappers in Generics & Streams

Generics* require objects:



4. Working With Characters

- Character wrapper objects:

- `Character.isDigit()`
- `Character.isLetter()`
- `Character.toUpperCase()`



Summary

- ✓ Collections
- ✓ Parsing
- ✓ Comparisons
- ✓ Character handling
- ✓ Streams & generics

Important Methods in Wrapper Classes (Simple & Useful)

Wrapper classes come with built-in methods that primitives do not have. These methods help convert data, compare values, check conditions, and more.

2. `valueOf()` Method

Returns a wrapper object



```
Integer x = Integer.valueOf(10);
```

```
Double d = Double.valueOf('5.6')
```

🕒 **Daily-life example** Like reading a label on a candy packet
→ converting printed text *25* to the actual number 25.

3. `toString()` Method

Converts a wrapper object → string



```
Integer price = 250;
```

```
String text = price.toString()
```

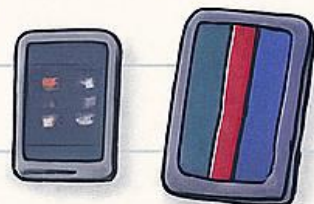
Putting something inside a box to make it easy to store

4. Checking Methods (Character class)

Useful for validating input

```
Character.isDigit('5'); // true
```

```
Character.isLetter('A'); // true
```



Checking heights using a measuring scale instead of guessing

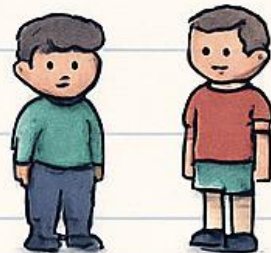
☑ Comparison Methods

Easier than using <, > with objects

```
Integer.compare(10, 20); // -1
```

```
Double.compare(2, 5, 1, 5); // 1
```

Picking the taller or



Why These Methods Matter?

✓ Input validation ✓ Safe comparisons

✓ Useful in forms, APIs, databases

shorter between two friends or letter,
SHORTER TALLER

Constructors in Wrapper Classes (Old vs New Java)

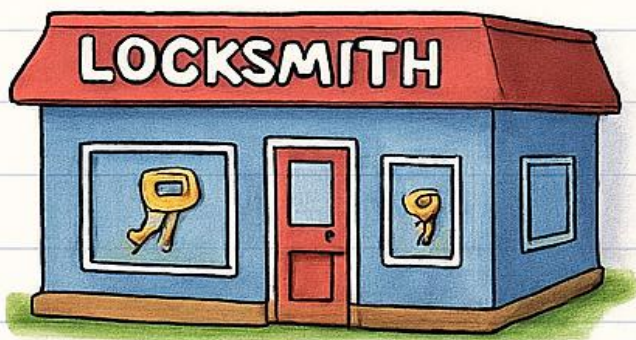
Wrapper classes originally had constructors, but Java discouraged using them after Java 9 because they were:

- ✓ Slower
- ✓ Confusing
- ✓ Not memory-efficient
- ✗ efficient

Now we use `valueOf()`, parsing methods, and autoboxing instead.



7. Oally-Life Example – Two Ways of Getting a House Key



Imagine needing a house key:

- Going to a locksmith* to make a new key every time.



1. Old Constructor Style (Deprecated)

`Integer a = new Integer(10);`
`Double b = new Double(5.6);`

- ✓ ✗ Fast, efficient

2. Modern Recommended style – `valueOf()`

`Integer a = Integer.valueOf(10);`
`Double b = Double.valueOf(5.6)`

- ☉ uses Integer Cache for values from -128 to 127.

3. 4. Modern Constructor (Deprecated – now))

- `Integer num = new Integer('25');` → Both x y will point to
- `Integer num = Integer.parseInt('25');` same memory.

→ `Integer x = Integer.valueOf();`
→ `Integer y = Integer.valueOf(10)`

Easy Memory Trick: Constructor = call locksmith (slow)
`valueOf()` = use spare key (fast)

Autoboxing vs Unboxing (Detailed & Easy Revision)

By now, we've used autoboxing and unboxing many times. This page gives a full revision so you never forget it again.

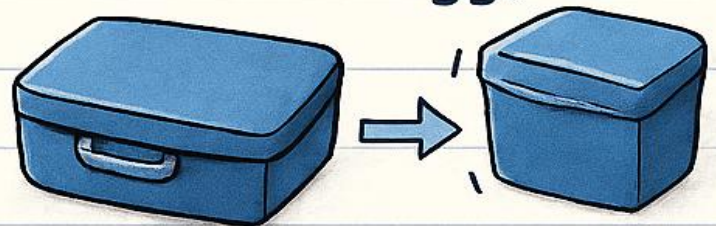
Daily-Life Example (Lunchbox Analogy)

- Primitive = Food

Plain, simple, usable as it is.

- Wrapper = Lunchbox

Adds features: carries food, keeps it covered, easy to store



↑ pack —————→ unpack
Autoboxing Unboxing

✦ Autoboxing

Putting food → inside a lunchbox

(primitive → wrapper automatically)



✦ Unboxing
Taking food out → from lunchbox

This analogy makes it very easy to remember!

Autoboxing (Primitive → Wrapper)

Java automatically converts a primitive value into a wrapper object.



- Example:

```
int price = 50;
```

```
Integer box = price; //autoboxing
```

Behind the scenes:

```
Integer box = Integer.valueOf(price);
```

⚠ Safe version:

```
int n = (num == null) ? 0 :
```

Daily-life analogy:

```
List<Integer> list =
```

```
new ArrayList<>();
```

```
list.add(5) = autoboxing
```

Daily-life analogy: If your lunchbox is empty → you can't eat anything! Wrapper = empty → unboxing fails

Memory Trick

- Auto-boxing → Java automatically gives you a lunchbox

Wrapper Class Caching (Integer Cache Explained)

Java stores certain wrapper objects in a special memory area called a cache. This helps Java save memory and improve performance.

The most famous one: Integer Cache.

Integer Cache (Most Important)

Creating new objects repeatedly is slow and wastes memory.



 Daily-Life Example – Ice Cream Sticks in a box

Imagine a shop owner:
keeps common flavours
(mango, vanilla, chocolate)

Java uses cache:

cached (pre-stored)



Rare values = new object

- Java does the same:

Common values = cached

- Rare values = new object

Rare values = new object



Example 1 (Cached $\Rightarrow$ same)

Example 2 Use `valueOf`

```
Integer a = 20;  
Integer b = 20;  
System.out.println(a == b)  
// true (same object)
```

```
Integer x = 300;  
Integer y = 300;  
System.out.println(x == y);  
// false (different objects)
```

 Example 3 (Using `valueOf`)

```
Integer p = Integer.valueOf(50);  
// = cached
```

`p == q`

Don't use $\neq$ to compare
wrapper values unless
you are 100% sure of caching
`a.equals(b)`.

 Memory Trick

- Cached = popular ice cream flavors kept ready: are caching
- Java uses cache to speed things up!

Byte Wrapper Class

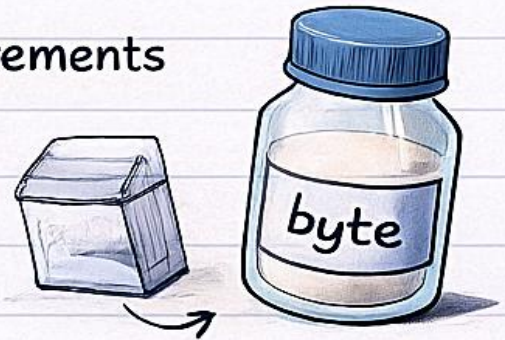
The **Byte** wrapper class represents the primitive **byte** value as an object.

Daily-Life Example — Small Sugar packets:

- Very small quantity
- Used when you need small measurements
- Fit easily in large storage boxes

→ Sugar packet = **byte**

→ Small labeled container = **Byte**



◆ Range of byte

-128 to 127

Lowest storage in Java (1 byte = 8 bits)

◆ Creating Byte Objects:

- `Byte b = Byte.valueOf((byte) 10);`
- Autoboxing: `Byte b = 10;` // autoboxed to `Byte`

◆ Useful Byte Methods

- `List<Byte>.println (Byte.MAX_VALUE);` // 127
- `System.out.println (Byte.MIN_VALUE);` /// -128

⚠ Warning: Overflow

`byte b = 127;` → becomes -128 (wraps around)

Filling a small sugar packet beyond limit → it spills and resets

Short Wrapper Class

(Simple & Complete Explanation)

- The Short wrapper class represents the primitive short as an object.
- short is slightly bigger than byte, still very memory-efficient.



 Tiny spoon = byte
 Small measuring cup = Short" object

Daily-Life Example – Measuring cups

Imagine small measuring cups used in a kitchen:



- A byte is like a tiny spoon
- A **short** is like a small measuring cup
- The cup: `Short.valueOf(120)`
- `Short s = 120;`

Range of short

-32,768 to 32,767

Bigger range than byte, still small in memory (2 byte).

Creating Short Objects.

`Short s = Short.valueOf(120)`

`Short s = 120;` / -auto boxed to Short

Useful Short Methods

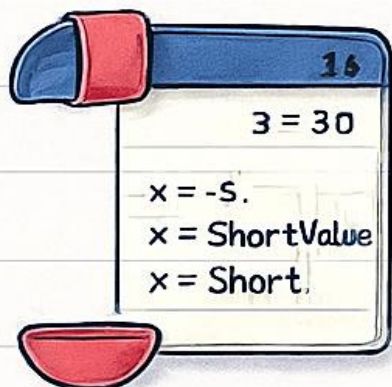
- ✓ `shortCValue()` – get primitive
- ✓ `parseShort(String)` – string →
- ✓ `compare(short, short)` numbers
- ✓ `MIN_VALUE` & `MAX_VALUE`

✓ `s.compareTo(ssort)`,

✓ `compare(short)t, s, (sShort12)`

⚠ Overflow Warning

Filling a measuring cup beyond its limit → it spills and resets



Memory Trick

- short = small measuring cup
- Short = small labeled measuring cup
- Used when values are "bigger than byte but smaller than int"

Integer Wrapper Class

(Most Used Wrapper in Java)

Integer is the wrapper class for the primitive `int`. It is the most commonly used wrapper in Java because numbers appear everywhere – counting, IDs, prices, scores, loops, collections.

Daily-Life Example : A Number on a Token Card

Think of:

- `int` = just a number (12)
- Written directly on paper.
- **Integer** = number stored on a laminated token card
 - cleaner
 - reusable
 - can be stored in a box
 - can be labeled
 - can be given to someone else easily

→ Paper with number = `int`
→ Laminated card = `Integer`



Paper with an `int` Laminated card = `Integer`

> Useful Integer Methods

Integer Integer Methods

```
Integer a = 50;    // -iree recommended
Integer a = 25;    // autornatic
Integer a = new Integer(25);  -- 1
```

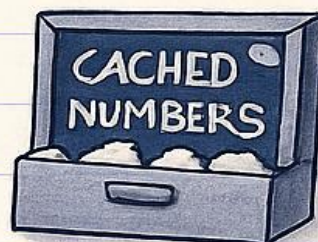


- Collections values (nullable)
- Database values (nullable) = `int`
- JSON, APIs, Spring, Hibernate objects, not primitives

> Useful Integer Methods

```
intValue()
Integer a = 50; → true
System.out.println(a == b);
→ true
```

Outside this range → new objects



- Memory Trick



Warning:
`int = raw number`
`int x = n.`

Use Integer when you need **null**, **number card** with features

Use Integer when you need **null**, collections, parsing, or framework

Double Wrapper Class

Real-Life Example

- Think of price tags at a store—they represent amounts that can include fractions of a whole number.



Double Class in Code

```
double primitivePrice = 19.99; // primitive
Double objecPrice = primitivePrice; // autobox
// Convert string
String priceString = "19.99";
Double price = Double.parseDouble(priceString);
System.out.println(price) // 19.99
```

When to Use Double

- Represent prices and measurements
- Handle null values in databases
- Work with frameworks/libraries expecting objects



Boolean wrapper class

Purpose

- The Boolean class wraps a boolean value (true or false) so it can be treated as an object.



Like how a light switch can be in an "on" or "off" state, a Boolean object can hold true or false.

```
Boolean isOn = Boolean.TRUE;  
boolean value = isOn.booleanValue();
```

Methods

- `parseBoolean` converts a String to a boolean
`boolean value = Boolean.parseBoolean(true)`
- `toString` returns the string "true" or "false"

Character Wrapper Class

Character is the wrapper class for the primitive *char*. A *char* stores one single character, but Character adds powerful utilities for checking, converting, and handling text.

Daily-Life Example – Alphabet Flashcard



The *char* is the printed letter → 'A'

The Character object is the flashcard with extra features:

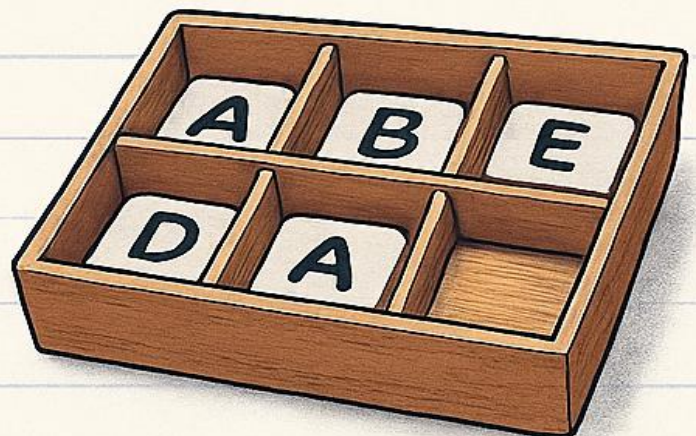
- ✓ Printed letter = `char`
- ✓ Flashcard with features = Character

Useful Scenarios

- ✓ Validating user input
(Only letters allowed? Only numbers?)
- ✓ Password checks
 - Must contain uppercase
 - Must contain digit
- ✓ Parsing character-by-character
(Useful in compilers, text editors, file parsers)
- ✓ Handling formatting
Uppercase/lowercase conversions.
- ✓ Reading text files
Processing each character safely.

Useful Character Methods

- ✓ Imagine you have a box full of letter tiles:
- ✓ Check if a tile is a letter →
- ✓ Check if it's uppercase → `toUpperCase`
- ✓ Check if tile to lowercase → `toLowerCase`, `isDigit`, `isLetter`



Important Warning

Character `ch` = 'AB'; // INVALID

Empty and null Wrapper

Empty Wrapper

`Integer a = new Integer("12345");` creates wrapper with a value of 12345.



`Integer b = new Integer("");` creates a wrapper with an empty value.

null Wrapper

`Integer c = null;` sets wrapper's value to null.



- Wrapper itself exists, but has no value

Why Use null?

- Allows variables to exist without a value yet (helps with logic)
- Indicates a 'missing' or uninitialized value

Double Wrapper Class

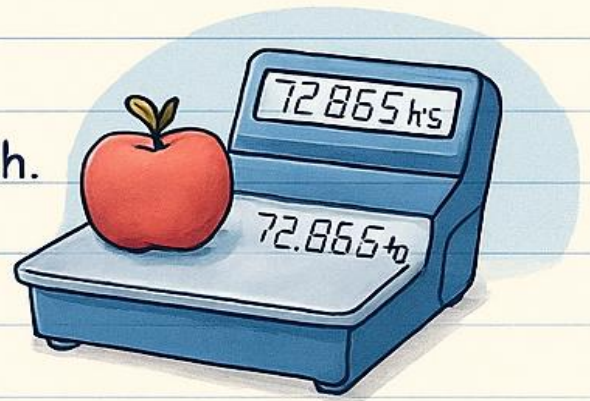
(Detailed Simple & Visual grade)

Double is the wrapper class for primitive double. It represents decimal numbers with double precision (15–16 digits).

It is the most accurate decimal type used in day-to-day Java math.

Daily-Life Example – Digital Weighing Machine

- double = the raw weight (72.865 kg)
- Double = weight shown + saved in machine memory



- compare weights
- store last reading
- check if value is valid
- display formatting

Creating Double objects

```
Double d  
= Double.valueOf(12.345);  
Autoboxing  
Double d = 12.345;
```

Useful Double methods

- doubleValue()
- parseDouble(String)
- compare(double, double)
- isNaN()
- isInfinite()

```
Double.MAX.VALUE  
Double.MIN.VALUE  
Double.MIN.NORMAL
```

Where Do We Use Double?

❌ Not for currency
Floating decimals lead to inaccuracies

Use BigDecimal for money.

Important Warnings:

double = high-precision weight
Double = weight stored in the smart weighing machine

Use Double when storing in collections, using frameworks, or allowing null.

Additional Utilities / Summary

- There are some other useful utilities available in Java.
For example:

Formatter - neat string formatting with specified patterns



Math.round - round a float/double to the nearest integer



Wrapper classes offer flexibility in using primitive values in an object-oriented way.

BigDecimal – The Safest Way to Handle Money & Precise Decimals

- BigDecimal is not a wrapper for a primitive, but it is the most important class when working with money, finance, high accuracy math.
- Float, double, and even wrapper classes <CANNOT handle exact decimals.

Daily-Life Example – Grocery Store Billing Machine

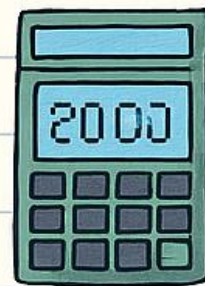


Imagine buying vegetables:

If the machine uses float/double → it may show ₹19.909999 instead of ₹20

— Float/Double = noisy weighing machine

🍃 BigDecimal = accurate digital billing machine



BigDecimal = noisy weighing machine

BigDecimal = accurate digital billing machine



BiviDecivl = divide 🌶️

Why Do We Need BigDecimal? (Simple Explanation)

✓ Synt & Double have rounding errors

System.out.println(0.1+0.2);
→ prints 0.300000000004 ✗

Important BigDecimal Methods (Easy to remember)

- Billing systems
- Bank transactions
- EMI calculations
- Tax calculators apps

✓ add (subtract)

due = total subtract (paid);

✓ multiply()

BigDecimal bill = rate
multiply (quantity);

✓ divide

amount.divide(divisor,
RoundingMode HALF.UP);

✓ compareTo()

if (amount.compareTo(limit)
> 0) {

✓ Important Note: Always use BigDecimal for ₹money
Float/Double = calculator with small inaccuracy

BigInteger – Working With Huge Numbers in Java

BigInteger is a special class used to handle numbers larger than what long can store.

A normal long can store up to:

9,223,372,036,854,775,807
(= 19 digits)

But BigInteger can store:

Unlimited digits 🤖

(Only limited by your RAM!)

Unlimited digits



unlimited ledger
= BigInteger

When Do We Use BigInteger?

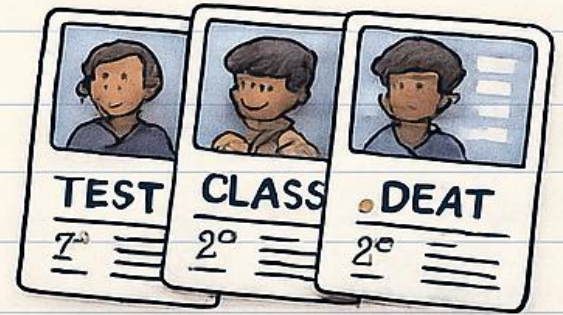
- ✓ **Cryptography (RSA encryption)**
Numbers can exceed 300+ digits
- ✓ **Blockchain, Bitcoin**
Very large integer arithmetic
- ✓ **Scientific simulations**
Star count, particle count, large factorials
- ✓ **Competitive programming**
Factorials, combinatorics



Number Class – The Parent of All Numeric Wrapper Classes

Daily-Life Example – “Master Template” for ID Cards

Imagine a school: Every student has an ID card. Different classes `<Integer>`, `<Long>` have their own card styles



This means all numeric wrappers share a common structure and common methods.

Why Does Java Use Number Class?

To give all numeric wrappers a common set of methods:

- ✓ `intValue()` `longValue()`
- ✓ `floatValue()` `doubleValue()`
- ✓ `shortValue()` `byteValue()`

Why This is Useful?

Easy type conversion

Any numeric wrapper can convert itself:

Integer n = 50;

	<u>Integer</u>
	<u>Float</u>
	<u>Long</u>
✓ <code>doubleValue()</code>	<u>Byte</u>
✓ <code>longValue()</code>	<u>Short</u>

Collections that store mixed numbers

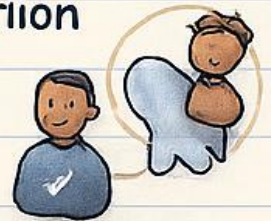
Example: `1sthuher > Florg`

Important Number Methods (Inherited by All Wrappers)



✓ Easy type conversion

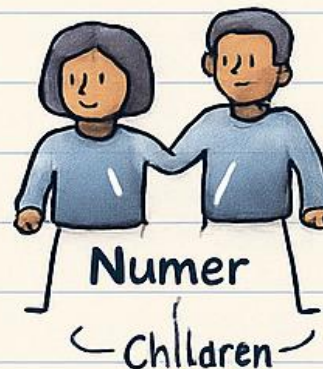
Any numeric wrapper can convert itself:



Integer n = 50;

double d = n.doubleValue();

✓ Polymorphism for numbers



Memory Trick

- Number = Parent
- Numeric = Children

Think of Number as master template, and all wrappers as different cards made from the same template.

Math Class

(Essential Utility for Numbers)



Math Class – Your All-in-One Calculator in Java

- Provides pre-built mathematical tools for: rounding
- absolute values power & roots
- random values max/min
- comparisons trigonometry
- logarithms



You never create a *Math* object – all methods are static.

Daily-Life Example – Scientific Calculator

- Think of: Wrapper classes = numbers you store *Math* class = calculator you use to operate on those numbers.
- Wrapper classes hold data.
Math performs operations on that data.



↑ `abs().asO // 5 in)` `Math.round(4.6)`
`Math.abs(-50); / 50` `// 5`

↑ `sqrt()` – square r `Math.ceil(4.1)` Ceil =
`Math.pow(2.5) / 32` `Math.floor(49)` round
round DOWN ↓



Like rounding the price of a vegetable to next rupee.

Character Class Methods

- Works with characters (char data type and Character wrapper class)

Real-Life Example

- Think of a ticket checker! They look at your ticket and tell you whether it's valid, where you can sit, etc.



Main Methods

isLetter: Checks if a character is a letter
`Character.isLetter('A') // true`

isDigit: Checks if a character is a number
`Character.isDigit('4') // true`

toUpperCase: Converts to to uppercase
`Character.toUpperCase('b') // 'B'`

toLowerCase: Converts to to lowercase
`Character.toLowerCase('Z') // 'z'`

More Character Class Methods

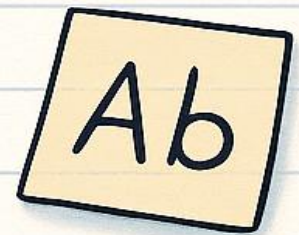
isWhitespace()

- Checks if a character is a whitespace



isAlphabetic()

- Checks if a character is a letter



compare()

- Compares two characters and returns a value



```
int result = Character.compare('a', 'b');
```

Other helpful methods:

- isUpperCase()
- isDigit()
- toLowerCase()

parseXxx() methods

Convert Strings containing numbers like "123" into actual primitive values (int, double, etc.)



- Clean up user input

- Parse Strings from files



- Convert text to



Example

```
String s = "123"; numbers
```

```
// String with digits
```

```
int num = Integer.parseInt(s);
```

```
// Converts to int: 123
```

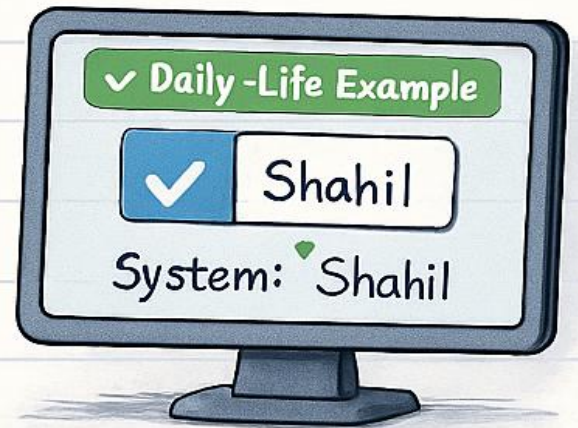
- Throws exception for invalid input ("abc")
- Different methods for each primitive type

Character Case Conversion Methods

Simple & Useful

Java provides handy methods to change the case of characters:

- `toUpperCase()` → Convert to UPPERCASE
- `toLowerCase()` → Capitalize the first letter (useful in names, titles)



Daily-Life Example – Name Formatting

⬆️ `.toUpperCase()`

Converts a character to uppercase:

```
char c = 'a';  
char result =  
Character.toUpperCase(c)
```

📌 Used in:

- Search bars
- Username standardization
- Serial numbers

⬇️ `.toLowerCase()`

Converts a character to title-case.

```
char c = 's';  
char result =  
Character.toLowerCase(c)
```

📌 Used in:

- Name formatting
- Contact lists



Memory Trick

- ✓ UP in "UPPERCASE"
- ✓ LOW in "LOWERCASE"
- ✓ Title of a book starts Capital
→ `toLowerCase()`

Why These Methods Are Important?

Digit & Number Handling With Character Class

Java provides useful methods to check if a character is a digit and to convert digit characters into actual numeric values.



Perfect for login forms, PIN checks, OTP inputs, and keypads.

- Key press = character
- Actual PIN value = number

For this conversion, Java uses Character's digit utilities.



1. isDigit()

Checks if a character is a digit (0-9)

```
char c = "7";  
Character.isDigit(c);  
// true
```

✓ Used in:

- OTP validation

2. getNumericValue()

Converts character → numeric integer

```
char c = "9",  
int num = Character  
getNumericValue(c); // 9
```

✓ Used when

converting keypad

Character: Alphabet & Letter Checking Methods

The Character class provides several tools to check:

- Is a character a letter?
- Is it alphabetic?
- Is it a letter or digit?
- Is it valid for names / usernames?

These are extremely useful in forms, signup pages, password rules, input validation, etc.



1 isLetter()

Checks if the character is a letter (A-Z or a-z)

```
Character.isLetter('A'); true  
Character.isLetter('3'); false
```

✚ Used in:

- ✓ Name validation
- ✓ Text-only input fields
- ✓ Detecting words in documents

Memory Trick:

- Letter? → isLetter
- Letter of ANY language?
- Letter OR Digit?
- → isLetterOrDigit

2 isAlphabetic()

Checks if the character is either a letter or a digit.

```
Character.isLetterOrDigit('');  
f*; true 🧠  
Character.isLetterOrDigit('');  
#; false
```

✚ Used in:

- ✓ International validation
- ✓ Unicode processing

4 isDefined()

Checks if the character exists in Unicode table

✚ Useful for:

- ✓ Preventing invalid symbols

Character: Whitespace, Control & Special Character Checks

Whitespace, Control & S Special Character Handling

These methods help Java understand:

- Is a character a SPACE?
- Is it a CONTROL character?
- Is it a SYMBOL or PUNCTUATION?

These are extremely important for real-world apps.



Daily-Life Example - WhatsApp/Instagram Chat

When you type a message:

- Leading spaces are ignored - whitespace
- Emojis & symbols are allowed
- Control characters (like hidden line breaks) are removed

2. `isSpaceChar()`

Checks only visible space characters, unlike `isWhitespace()`.

2. `isSpaceChar()`

Checks only *'visible'* space characters, unlike

```
Character.isWhitespace(' ');  
// true  
Character.isWhitespace('\n');  
true
```

✓ Used in:

- Cleaning user input
- Removing extra spaces
- Ignoring blank characters in text

3. `isISOControl()`

Checks if the character is a *control character* (non-printable).

```
char c = ' ';  
if(Character.isSpaceChar(' '));  
// false  
Character.isSpacechar('@');  
// true
```

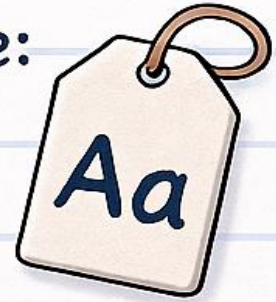
Special Character Check (Custom logic:

- Password rules
- Symbol filtering
- Chat apps emoji detection

Character: Unicode, Case & Identifier Rules



Daily-Life Example:
Naming a Baby
or Username

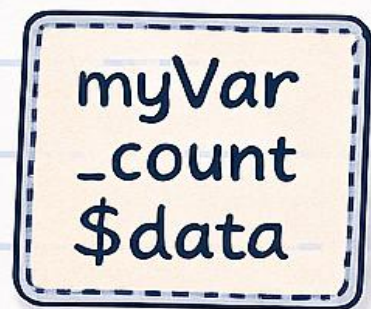


`isJavaIdentifierStart()`: Can it start
a character a name?

Aa

`isJavaIdentifierPart()`:
Can it appear inside
a name

`isUpperCase()` &
`isLowerCase()`
Check if a character
is uppercase or lowercase



Memory Trick

Start → `isJava` /=
Middle → `isLow` /=
Case → `isTitleCas`

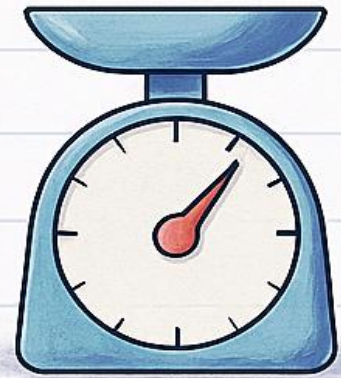
`getType()`: Understand
the Unicode category

→ `isJavaIdentifierStart()`
→ `isJavaIdentifierPart()`

Wrapper Classes in Java

BigDecimal

- Used for precise decimal calculations
- Overcomes limitations of float/double



Example Use:

```
class Recipe {  
    public static void(String[] args) {  
        BigDecimal amount = Big.valueOf(0.2);;  
        System.out.printf("Sugar = %s cups\n\n")  
    } amount  
}
```

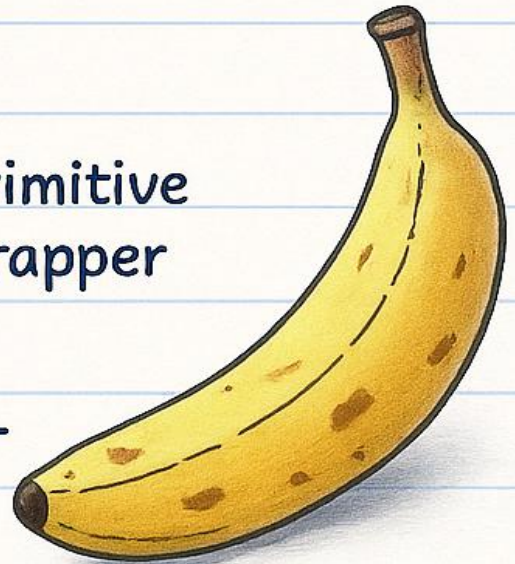
Used to measure ingredients in a recipe with complete accuracy (no rounding)



Autoboxing and Unboxing

What are Autoboxing and Unboxing?

- Autoboxing: Convert a primitive into its corresponding wrapper class (int → Integer)
- Unboxing: Java automatically convert a wrapper class back to its corresponding primitive → int)



Real-Life Example

- To: packing a banana for travel, then unboxing it before eating = packing

Java Code Example

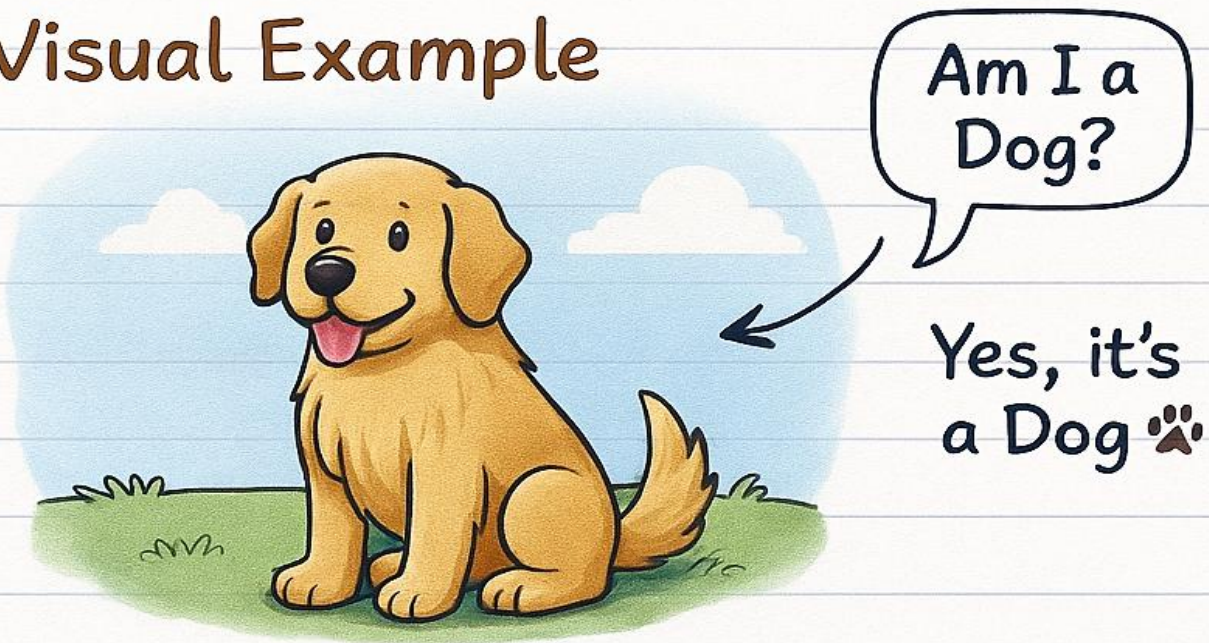
```
Integer numObj = 42;    // autoboxing  
int num = numObj;       // unboxing
```


instanceof keyword

What is instanceof?

- Test if an object belongs to a sub-class
- Return true or false

Visual Example



Code Example

```
public Dog d;  
public Dog extends Animal {  
    void mainSouma ("doWoof!");  
    if (d instanceof Dog ()) {  
        } system.out.prn("t's a Dog");  
    }  
}
```


Autoboxing in Java

Autoboxing Meaning

- Autoboxing is the automatic conversion of a primitive to its wrapper class.



Real-Life Example

- Like pouring coffee into a paper cup – the coffee (primitive value) is automatically wrapped (placed in object).

How It Works

- When you assign a primitive to a wrapper class, Java handles the conversion for you.

```
int num = 5;
```

```
Integer obj = num; // autoboxing
```

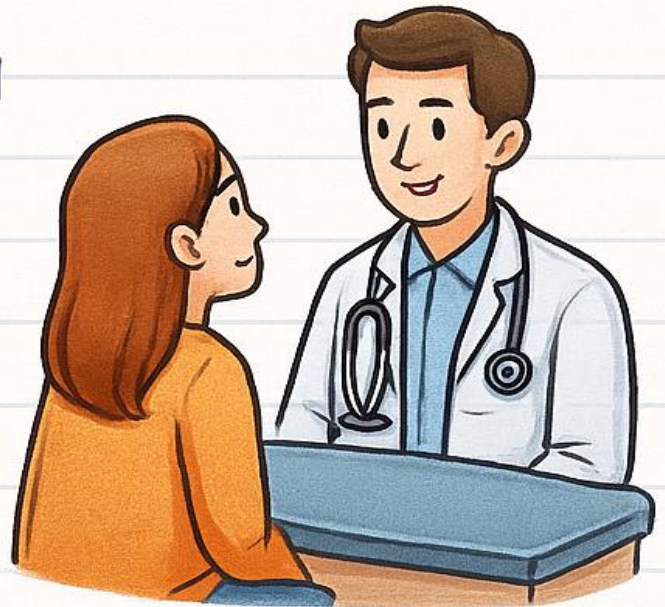

PriorityQueue class

What does it do?

- It stores elements in a way that orders higher priority elements first

Real-Life Analogy

- Think of an emergency room: patients with more urgent needs (high priority) see the doctor first



Example

```
PriorityQueue<Integer> q =  
    new PriorityQueue(>.;  
    q.add(60); // Add elements  
    q.add(20);  
    q.add(40)  // Remove highest  
                priority (20)
```


Wrapper Classes in Java

Cheat Sheet

Integer

- .compare ()
- .valueOf ()
- .intValue ()
- .parseInt ()
- .toString ()
- .MAX_VALUE
- .MIN_VALUE

Byte

- .valueOf ()
- .byteValue ()
- .parseByte ()
- .toString ()
- .MAX_VALUE
- .MIN_VALUE

Boolean

- .valueOf ()
- .booleanValue ()
- .parseBoolean ()
- .toString ()

Float

- .compare ()
- .valueOf ()
- .floatValue ()
- .parseFloat ()
- .toString ()
- .MAX_VALUE
- .MIN_VALUE

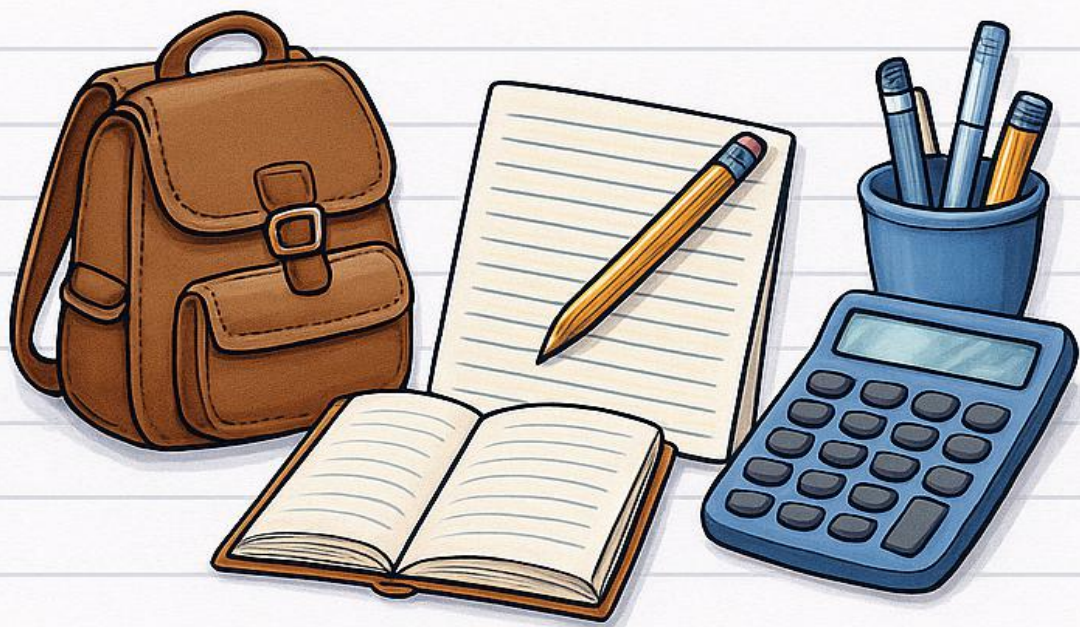
Double

- .valueOf ()
- .booleanValue ()
- .parseBoolean ()
- .toString ()

Character

- .isLetter ()
- .isDigit ()
- .isWhitespace ()
- .isUpperCase ()
- .isLowerCase ()
- .getType ()

“Keep learning, keep growing.”



 Shahil Raghuwanshi

 raghuwanshisahil21@gmail.com

 github.com/ShahilRaghuwanshi

The End